MODELO DE PROPENSIÓN DE COMPRA

Documentación Técnica de la Solución

Ferreycorp, Prueba Técnica de Selección

Abril 2026

Nombre: Milagros Valeria Gómez Salcedo

DNI:75317982

Machine Learning	Pricing	GenAI	Storytelling	Cloud Computing

SOLUCIÓN EN PRODUCCIÓN, PRUEBA FUNCIONAL

Frontend (Streamlit)	https://propension-frontend-827318632719.us-east1.run.app/
Backend API (Swagger)	https://propension-backend-827318632719.us-east1.run.app/docs

Ambas URLs están en producción en Google Cloud Run y disponibles para prueba funcional.

Resumen Ejecutivo

Este documento consolida el trabajo completo de desarrollo e implementación del sistema de Propensión de Compra para Ferreycorp: desde el análisis exploratorio de datos hasta el despliegue en producción en Google Cloud, pasando por el modelado, la validación rigurosa, la integración de un agente de IA conversacional y la construcción de un frontend web.

La solución responde a la pregunta central del negocio:

"¿A quién contactar hoy para maximizar las ventas con el menor esfuerzo?"

La arquitectura resultante combina machine learning, analítica en la nube, un agente LLM y una interfaz web operativa, todo desplegado y accesible en producción.

Indicador clave	Valor	Significado para el negocio
AUC Warm-Start (test temporal)	0.685	Poder discriminatorio real en condiciones de producción
AUC Cold-Start (CV 5-fold)	0.746	Discrimina por demografía en primera visita sin historial
Lift decil 10	2.34x	El top 10% por score compra 2.3x más que el promedio
Uplift en llamadas comerciales	+84%	Llamar a clientes ALTA genera 84% más conversiones
Clientes ALTA prioridad	72 de 500	Candidatos para llamada esta semana
Costo mensual en GCP	~$7/mes	BigQuery + Vertex AI + Cloud Run con créditos GCP
Estabilidad del modelo	±0.0006 AUC	Resultado idéntico con cualquier semilla aleatoria

Cobertura de Skills Evaluados

A continuación se detalla cómo cada skill evaluado está cubierto por la solución construida:

Skill	Cómo se cubre	Evidencia
Machine Learning	Arquitectura dual cold-start/warm-start. 4 algoritmos comparados. Backtesting rolling. Fine-tuning. Permutation Importance + SHAP.	AUC 0.685 temporal, lift 2.34x, estabilidad ±0.0006
Pricing	Análisis del ROI por canal (email vs llamada vs CRM). Costo ~$7/mes en GCP. Estrategia de contacto diferenciada por umbral.	Uplift +84% en llamadas, +48% en email
GenAI	Agente conversacional sobre Vertex AI con Gemini 2.5 Flash. Patrón planner → SQL → respuesta. 3 tools (query_scores, explain_client, predecir_cliente).	URLs de producción funcionales
Storytelling	EDA con distinción conteo vs tasa. Tabla decil-lift. KPIs comerciales accionables. Documentación en lenguaje de negocio.	Este documento y la presentación PDF
Cloud Computing	BigQuery, Cloud Storage, Cloud Run, Vertex AI, Docker, FastAPI, Streamlit. Pipeline completo de 15 pasos.	2 servicios desplegados y operativos en GCP

1. Objetivo del Proyecto

1.1 Objetivo General

Desarrollar e implementar un sistema de Propensión de Compra que prediga si un cliente comprará o no un producto en su próxima visita, integrando los resultados con un agente conversacional de IA que permita al equipo comercial consultar y filtrar predicciones en tiempo real usando lenguaje natural, desplegado en Google Cloud Platform.

1.2 Objetivos Específicos

- Analizar los datos históricos de 500 clientes en 730 días para identificar patrones de comportamiento de compra.
- Construir un pipeline de features que capture señales predictivas sin violar la secuencia temporal de los datos.
- Seleccionar y validar el mejor modelo mediante backtesting rolling, replicando las condiciones reales de producción.
- Resolver el problema del cliente nuevo (cold start) con un modelo demográfico independiente.
- Desplegar los scores en BigQuery (Google Cloud) y crear una vista de negocio para acceso optimizado.
- Construir un backend FastAPI con endpoints claros para predicción, consulta y chat.
- Integrar un agente de IA sobre Vertex AI con Gemini 2.5 Flash.
- Construir un frontend Streamlit con dashboard, chat conversacional y simulador de predicción.
- Dockerizar y desplegar ambos servicios en Cloud Run.
- Cuantificar el impacto económico del modelo en términos de KPIs accionables.

2. Análisis Exploratorio de Datos (EDA)

2.1 Descripción del Dataset

El dataset compras_data.csv contiene 58,693 visitas de 500 clientes únicos durante 730 días. Cada fila representa una visita con información sobre si compró, precios vigentes, promociones y características sociodemográficas.

Dimensión	Valor
Filas totales	58,693 visitas
Clientes únicos	500
Período	730 días (~2 años)
Tasa de compra global	24.94%
Tasa de compra, primera visita	10.6% (diferente al global, justifica dos modelos)
Desbalance general	3:1 (no compra vs compra)
Desbalance primera visita	8.4:1 (mucho mayor, requiere tratamiento especial)
Marcas disponibles	5 marcas de producto
Valores nulos	0 (dataset limpio, sin necesidad de imputación)
Variable objetivo	incidencia_compra (0=no compró, 1=compró)
Variable excluida	tamanio_ciudad (por instrucción del diccionario de datos)

2.2 Hallazgos Principales

Hallazgo 1: El comportamiento pasado predice el futuro

El hallazgo más importante del EDA fue que las correlaciones lineales entre las variables crudas y la compra son casi nulas (máximo 0.074 para edad). Sin embargo, las variables derivadas del historial muestran gradientes fuertes y monótonos:

Variable derivada	Valor bajo → tasa compra	Valor alto → tasa compra
Tasa histórica 0–10%	11.7%	40–100% → 58.3%
Días sin comprar >60d	11.5%	1–7 días → 40.8%
Compras en 30 días = 0	13.3%	3+ compras → 51.0%

> *El principio central del modelo: quien compró frecuentemente antes, comprará frecuentemente después. La correlación entre tasa_historica personal y el score final del modelo es 0.814.*

Hallazgo 2: Alta lealtad de marca

El 79.9% de los clientes repite la misma marca en visitas consecutivas. La Marca 5 lidera ventas (34% del volumen) y tiene la mayor tasa de recompra post-compra (56.5%). La variable ultima_marca_comprada es el predictor más potente del modelo.

Hallazgo 3: La distinción crítica, conteo vs tasa de compra

Un análisis superficial (como el que hace Power BI con SUM por defecto) muestra que el grupo 26-35 años genera más compras. Sin embargo, esto se debe a que hay más clientes jóvenes, no a que cada uno compre más. La métrica correcta para el modelo es la tasa individual (media), no el conteo absoluto.

Grupo edad	Clientes	Tasa de compra	ROI simulado*
18-25	61	20.0%	100% (referencia)
26-35	182	22.3%	123%
36-45	126	26.9%	169%
46-55	71	27.3%	173%
56-75	60	30.6%	206% (mejor ROI)

* ROI simulado: costo contacto=1 unidad, valor compra=10 unidades.

Hallazgo 4: Primera visita, problema diferente

La tasa de compra en primera visita es solo 10.6%, frente al 24.9% general. El desbalance es 8.4:1, no 3:1. Estos son dos problemas fundamentalmente distintos que requieren modelos separados (cold-start y warm-start).

Hallazgo 5: Análisis de marcas y promociones

Cinco segmentos de precio bien diferenciados (Marca 1: $1.39 → Marca 5: $2.65). La Marca 5 tiene el mayor uplift promocional (+6.9 pp). La Marca 3 es insensible a promociones (+0.7 pp), sus compradores actúan por hábito, no por precio.

Hallazgo 6: Truncamiento en semana 105

La semana 105 contiene solo los días 729-730 del dataset, con 0 compras registradas. Este es un efecto de truncamiento temporal, no una tendencia real del negocio. Esta semana fue excluida de los análisis temporales para evitar sesgos.

Hallazgo 7: Estructura longitudinal, implicaciones de diseño

El dataset no debe tratarse como una tabla IID (independiente e idénticamente distribuida). Hay dependencia temporal, historial comportamental y riesgo de leakage si se mezcla información del futuro con el presente. Esto obligó a diseñar features históricas y a usar backtesting rolling en lugar de split aleatorio.

3. Procesamiento y Limpieza de Datos

3.1 Limpieza

- Variable tamanio_ciudad excluida por instrucción del diccionario de datos.
- Cero valores nulos, sin necesidad de imputación.
- Variables id_marca y cantidad del evento actual excluidas como features (describen el evento que se quiere predecir).
- Dataset ordenado por cliente y luego por día de visita, requisito fundamental para el feature engineering temporal.

3.2 Feature Engineering, La regla anti-leakage

Se construyeron 13 variables derivadas en cuatro grupos. La regla crítica que rige todo el proceso:

> *Para calcular el feature del día T, se usa SOLO información de los días 1 a T-1. Si se usara información del día T también, el modelo vería su propia respuesta como predictor → AUC artificialmente inflado → fallo en producción.*

Grupo	Variables	Qué capturan
Comportamiento histórico	tasa_compra_historica, dias_desde_ultima_compra, compras_acumuladas, frecuencia_30d, compras_3visitas, habia_comprado_ayer	Cuán frecuente y reciente es el comportamiento de compra del cliente
Contexto de mercado	precio_minimo_dia, precio_promedio_dia, rango_precios_dia, num_promos_activas	Qué tan atractivo está el mercado ese día
Relación cliente-mercado	precio_ultima_marca, promo_sobre_ultima_marca, precio_relativo_ultima	¿Está barata la marca favorita del cliente hoy?
Temporales	dia_visita, trimestre, semana	Estacionalidad y ciclos de compra

3.3 Segmentación RFM

Se construyó una segmentación K-Means K=4 usando Recencia, Frecuencia y Valor. El segmento se calcula SOLO sobre los datos de entrenamiento de cada ventana temporal y se aplica al test, evitando que el clustering use información del futuro.

Segmento	Perfil	Tasa compra promedio
Alto (3 clientes)	Compran en casi toda visita, recientes	92%
Medio-Alto (77 clientes)	Compradores frecuentes activos	50%
Medio-Bajo (380 clientes)	Compradores ocasionales	19%
Bajo (39 clientes)	Compradores esporádicos o inactivos	12%

4. Metodología de la Solución

4.1 Por qué el backtesting rolling es la única validación correcta

Los datos son longitudinales (cliente-tiempo). Usar un split aleatorio (train_test_split estándar) mezcla visitas futuras en el entrenamiento: el modelo 'aprende' comportamientos que en la realidad aún no habrían ocurrido. El resultado es un AUC inflado que falla en producción.

La validación correcta es el backtesting rolling con 3 ventanas temporales más un test final intocable:

Ventana	Entrenamiento	Validación	Obs. train
Ventana 1	Días 1-365	Días 366-450	31,080
Ventana 2	Días 1-450	Días 451-540	37,788
Ventana 3	Días 1-540	Días 541-620	44,710
Test FINAL	Días 1-620	Días 621-730 (NUNCA VISTO ANTES)	50,556

> *El test final (días 621-730) nunca fue visto durante el backtesting ni el fine-tuning. Es la estimación más limpia del rendimiento real del modelo en producción.*

4.2 Arquitectura de dos modelos, cold-start y warm-start

El descubrimiento de que la tasa de compra en primera visita (10.6%) es radicalmente diferente a la general (24.9%) llevó a diseñar dos modelos especializados:

Dimensión	Modelo Cold-Start	Modelo Warm-Start
Cuándo se usa	Primera visita (visit_rank = 0)	2da visita en adelante
Algoritmo	Regresión Logística (L2, C=0.05)	Random Forest (400 árboles)
Features	6 demográficas + 14 de mercado	35 variables: comportamiento + demografía + RFM
AUC	0.746 (CV 5-fold)	0.685 (test temporal)
Tasa basal	10.6%	25.1%
Umbral de decisión	0.15	0.43
Por qué este algoritmo	Más estable con solo 500 obs y desbalance 8.4:1	Bootstrap averaging amortigua el 'future shift' temporal

5. Desarrollo del Modelo de Propensión de Compra

5.1 Comparación de 4 algoritmos

Se evaluaron cuatro algoritmos en el backtesting rolling. Los resultados con validación temporal real:

Modelo	AUC CV	AUC Test Temporal	AP Test	Veredicto
Logistic Regression	0.731 ± 0.005	0.708	0.553	Baseline sólido
Random Forest	0.784 ± 0.005	0.708	0.562	CHAMPION ✓
HistGradientBoosting	0.816 ± 0.006	0.666	—	DESCARTADO
CatBoost	0.815 ± 0.004	0.676	~0.55	Challenger

Por qué se descartó HistGradientBoosting

HistGBM obtuvo el AUC más alto en validación cruzada (0.816), pero al evaluarse en el test temporal cayó a 0.666: una diferencia de 15 puntos porcentuales. Esto indica overfitting temporal, el modelo memorizó patrones específicos del período de entrenamiento que no generalizan hacia adelante. Es el mejor modelo 'en laboratorio', pero el peor 'en producción'. Esta caída de 15 pp no es ruido estadístico: es evidencia de que el modelo no es apto para uso real.

Por qué CatBoost quedó como challenger y no como campeón

CatBoost es teóricamente el más adecuado para este dataset porque maneja variables categóricas nativas (marca, género, estado civil). Sin embargo, bajo validación temporal obtuvo AUC 0.676 vs 0.708 de Random Forest. La recomendación es re-evaluarlo con más datos y mayor fine-tuning antes de promoverlo como campeón.

Por qué Random Forest ganó

Random Forest es más robusto al 'future shift' temporal porque promedia 400 árboles entrenados en muestras bootstrap independientes. Cuando el comportamiento de los días 621-730 difiere ligeramente del aprendido en 1-620, ese efecto se amortigua. La diferencia entre AUC CV (0.784) y AUC test (0.708) es de solo 7.6 puntos, frente a los 15 de HistGBM.

5.2 Fine-tuning y estabilidad

- Mejor configuración: n_estimators=400, max_depth=12, min_samples_leaf=20 (evaluada con StratifiedKFold 5-fold en train completo).
- Estabilidad verificada con 5 semillas aleatorias: AUC 0.685 ± 0.001, modelo completamente reproducible.
- Prueba formal con/sin RFM en backtesting rolling: el segmento RFM mejoró AP en 2 de 3 ventanas → SE QUEDA.

5.3 Explicabilidad, Permutation Importance + SHAP

Se usaron dos métodos complementarios para explicar el modelo:

- Permutation Importance: mide cuánto cae el AUC si se aleatoriza cada variable. Top 3: tasa_compra_historica, segmento_RFM, frecuencia_compras_30d.
- SHAP (SHapley Additive exPlanations): descompone cada predicción individual en contribuciones por variable, permitiendo explicar un score específico al equipo comercial.

> *Hallazgo clave de explicabilidad: la correlación entre tasa_compra_historica y el score final es 0.814. El modelo es esencialmente un refinamiento sofisticado del principio intuitivo 'quien compró antes, comprará después'. Esto lo hace más confiable, más explicable y más fácil de monitorear en producción.*

5.4 Métricas finales del modelo

Métrica	Warm-Start (RF)	Cold-Start (LR)
AUC-ROC	0.685 (test temporal)	0.746 (CV 5-fold)
Average Precision	0.464 (1.8x el baseline)	—
F1-Score	0.467 (umbral 0.43)	—
MCC	0.236	—
Lift decil 10	2.34x	—
Estabilidad	±0.0006 (5 semillas)	—

6. Arquitectura Cloud y Despliegue en GCP

6.1 Decisiones de arquitectura

La arquitectura final se apoyó en servicios administrados de Google Cloud para reducir fricción operativa. Cada decisión respondió a una necesidad específica:

Componente	Tecnología elegida	Por qué
Artefactos del modelo	Cloud Storage (GCS)	Desacopla el entrenamiento del servicio. El backend descarga los .pkl al iniciar sin tenerlos embebidos en el repositorio.
Scores de clientes	BigQuery	El patrón de consumo es analítico (filtros, rankings, agregaciones), no transaccional. Responde rápido a consultas complejas.
Vista de negocio	vw_scores_clientes (BigQuery view)	Expone solo las columnas necesarias con canal_sugerido ya calculado. Reduce complejidad para el agente.
Backend API	FastAPI	Validación de payloads, documentación automática Swagger, separación clara de responsabilidades por endpoint.
Frontend	Streamlit	Velocidad de implementación. Permite demo operativa con dashboard, chat y simulador sin overhead frontend.
LLM conversacional	Gemini 2.5 Flash (Vertex AI)	Credenciales unificadas con GCP, despliegue empresarial, integración nativa con el stack.
Empaquetado	Docker	Portabilidad garantizada. El mismo artefacto corre igual en desarrollo y en nube.
Despliegue	Cloud Run	Serverless, no administrar VMs. Escala automático. Integración con Artifact Registry.

6.2 Pipeline de despliegue, 15 pasos

1. Crear bucket en Cloud Storage (GCS).
2. Subir artefactos del modelo: modelo_cold_start.pkl, modelo_warm_start.pkl, scaler_rfm.pkl, kmeans_rfm.pkl, model_stats.json.
3. Subir scores de los 500 clientes a BigQuery (tabla scores_clientes).
4. Crear vista de negocio vw_scores_clientes con campos canal_sugerido, prioridad y recencia calculados.
5. Construir backend FastAPI con endpoints: /health, /top-clients, /predict, /filter-clients, /chat.
6. Implementar lógica de predicción cold/warm con detección automática por visit_rank.
7. Implementar las 3 tools del agente: query_scores, explain_client, predecir_cliente.
8. Integrar Gemini 2.5 Flash via Vertex AI con patrón planner → SQL → respuesta.
9. Probar backend en local con gcloud auth application-default login.
10. Dockerizar backend con requirements.txt y comando de arranque Uvicorn.

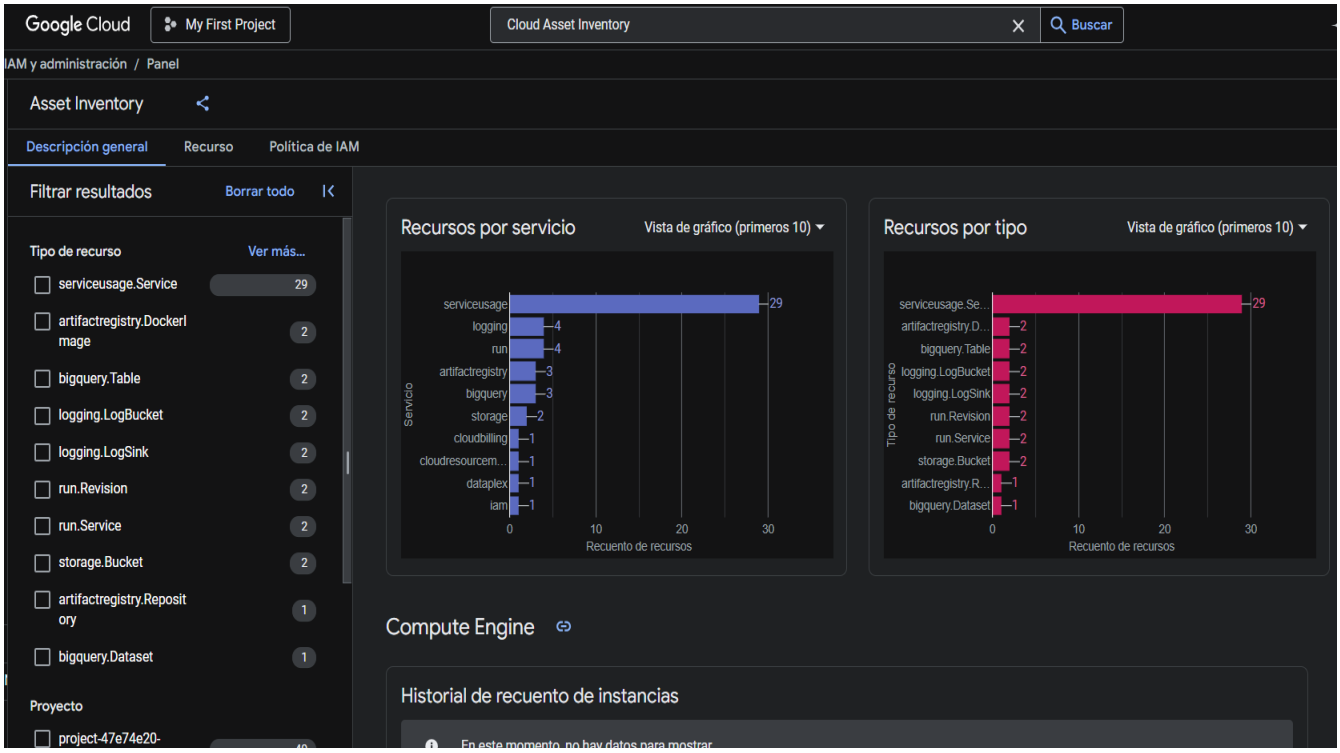
11. Construir frontend Streamlit con Dashboard, Chat y módulo Predicción.
12. Conectar frontend al backend via variable de entorno BACKEND_URL.
13. Dockerizar frontend con Streamlit y configuración de servidor.
14. Desplegar backend a Cloud Run via Cloud Build → Artifact Registry → Cloud Run.
15. Desplegar frontend a Cloud Run apuntando a la URL del backend en producción.

6.3 Endpoints del backend

Endpoint	Método	Propósito	Consumo
GET /health	GET	Verificar disponibilidad	Monitoreo
GET /top-clients	GET	Top clientes por score	Dashboard
POST /predict	POST	Scoring de visita nueva (cold/warm)	Simulador
POST /filter-clients	POST	Filtros estructurados	Dashboard y chat
POST /chat	POST	Chat conversacional con Gemini	Agente IA

6.4 Problemas encontrados y cómo se resolvieron

Problema	Resolución
Autenticación GCP (ADC) no encontraba credenciales	gcloud auth application-default login + configurar quota project explícito
Proyecto no detectado por librerías GCP	gcloud config set project + pasar project=PROJECT_ID a cada cliente
to_dataframe() requería db-dtypes	pip install db-dtypes pyarrow en requirements.txt
Gemini devolvía respuesta vacía sin error	Aumentar max_output_tokens, extraer texto desde candidates[].content.parts
Tools del SDK Gen AI con formato incorrecto	Patrón planner → SQL → respuesta en lugar de tool calling nativo
Docker no disponible en otra máquina	Instalar Docker como dependencia de sistema (no del venv)
Cloud Build con errores de permisos	Activar APIs requeridas y revisar roles IAM en el proyecto


Google Cloud
My First Project
Cloud Asset Inventory
Buscar
IAM y administración / Panel
Asset Inventory
Descripción general
Recurso
Política de IAM
Filtrar resultados
Borrar todo
Tipo de recurso
Ver más...
serviceusage.Service 29
artifactregistry.DockerImage 2
bigquery.Table 2
logging.LogBucket 2
logging.LogSink 2
run.Revision 2
run.Service 2
storage.Bucket 2
artifactregistry.Repository 1
bigquery.Dataset 1
Proyecto
project-47e74e20-
Recursos por servicio
Vista de gráfico (primeros 10)
Servicio
serviceusage 29
logging 4
run 4
artifactregistry 3
bigquery 3
storage 2
cloudbilling 1
cloudresourcem... 1
dataplex 1
iam 1
0 10 20 30
Recuento de recursos
Recursos por tipo
Vista de gráfico (primeros 10)
Tipo de recurso
serviceusage.Se... 29
artifactregistry.D... 2
bigquery.Table 2
logging.LogBucket 2
logging.LogSink 2
run.Revision 2
run.Service 2
storage.Bucket 2
artifactregistry.R... 1
bigquery.Dataset 1
0 10 20 30
Recuento de recursos
Compute Engine
Historial de recuento de instancias
En este momento, no hay datos para mostrar

7. Desarrollo e Integración del Agente de IA

7.1 Arquitectura del agente

El agente conversacional permite al equipo comercial interactuar con los resultados del modelo usando lenguaje natural, sin necesidad de conocer SQL ni Python. La lógica central usa el patrón planner → SQL → respuesta:

- El usuario escribe una pregunta en lenguaje natural.
- En la primera llamada, Gemini devuelve un plan estructurado indicando si necesita consulta SQL y cuál.
- El backend valida la SQL (solo SELECT, tabla completa obligatoria) y la ejecuta en BigQuery.
- En la segunda llamada, Gemini transforma el resultado en una respuesta ejecutiva con recomendación de acción.

7.2 Las tres tools del agente

Tool	Cuándo se usa	Modelo activado
query_scores(sql)	Clientes existentes con historial en BigQuery	BigQuery (vw_scores_clientes)
explain_client(id)	Cuando el vendedor pregunta por un cliente específico con ID	BigQuery + explicación en lenguaje natural
predecir_cliente(perfil)	Cliente nuevo o perfil hipotético sin ID conocido	Cold-Start o Warm-Start automático según visit_rank

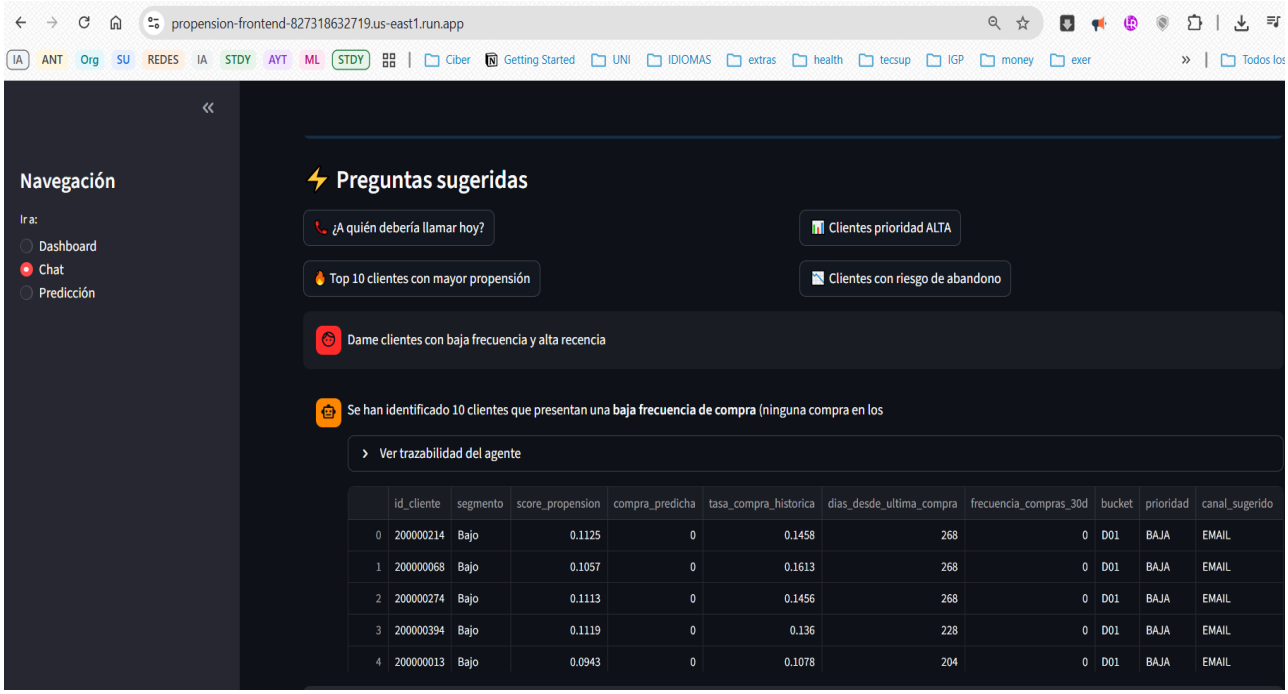
7.3 Lógica de selección de modelo en el agente

- visit_rank = 0 (primera visita): el agente extrae datos demográficos de la descripción del vendedor y activa Cold-Start.
- visit_rank > 0 (tiene historial): el agente extrae tasa histórica, recencia y frecuencia, y activa Warm-Start.

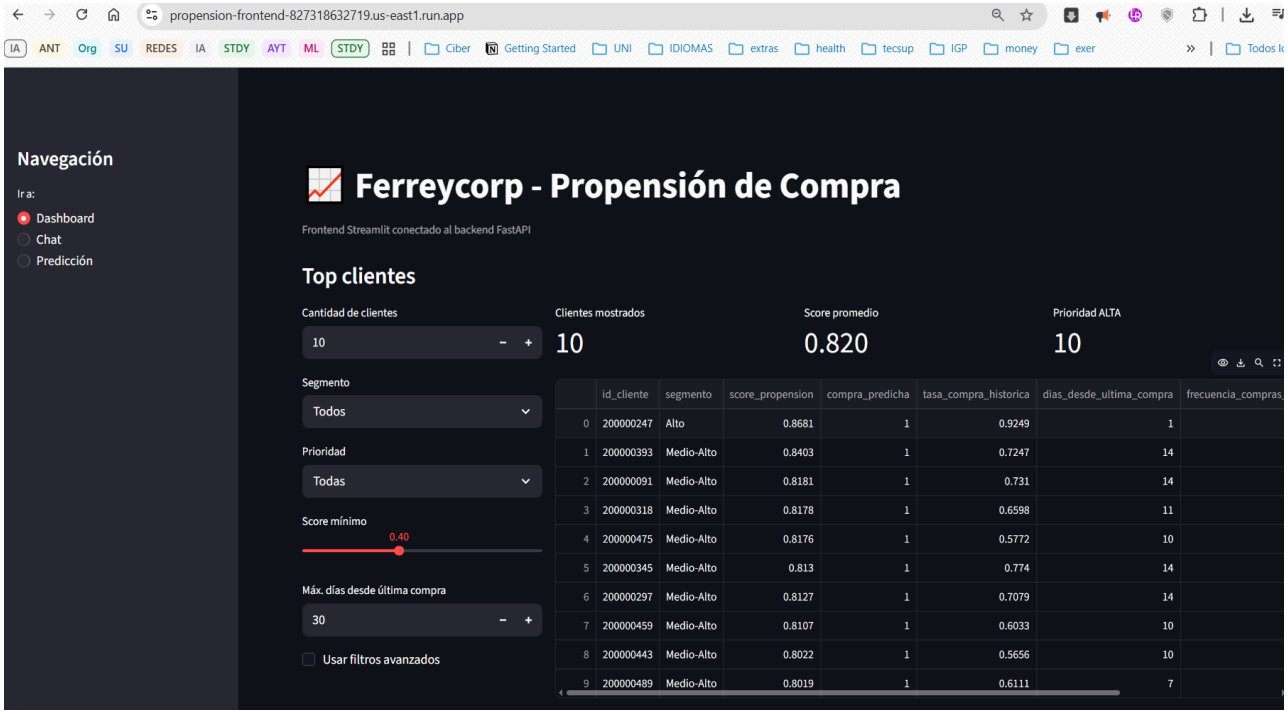
7.4 Ejemplos de consultas que el agente responde

Pregunta del vendedor	Tool	Modelo
¿A quién llamo hoy?	query_scores	BigQuery
¿Por qué el cliente 200000247 tiene score alto?	explain_client	BigQuery + explicación

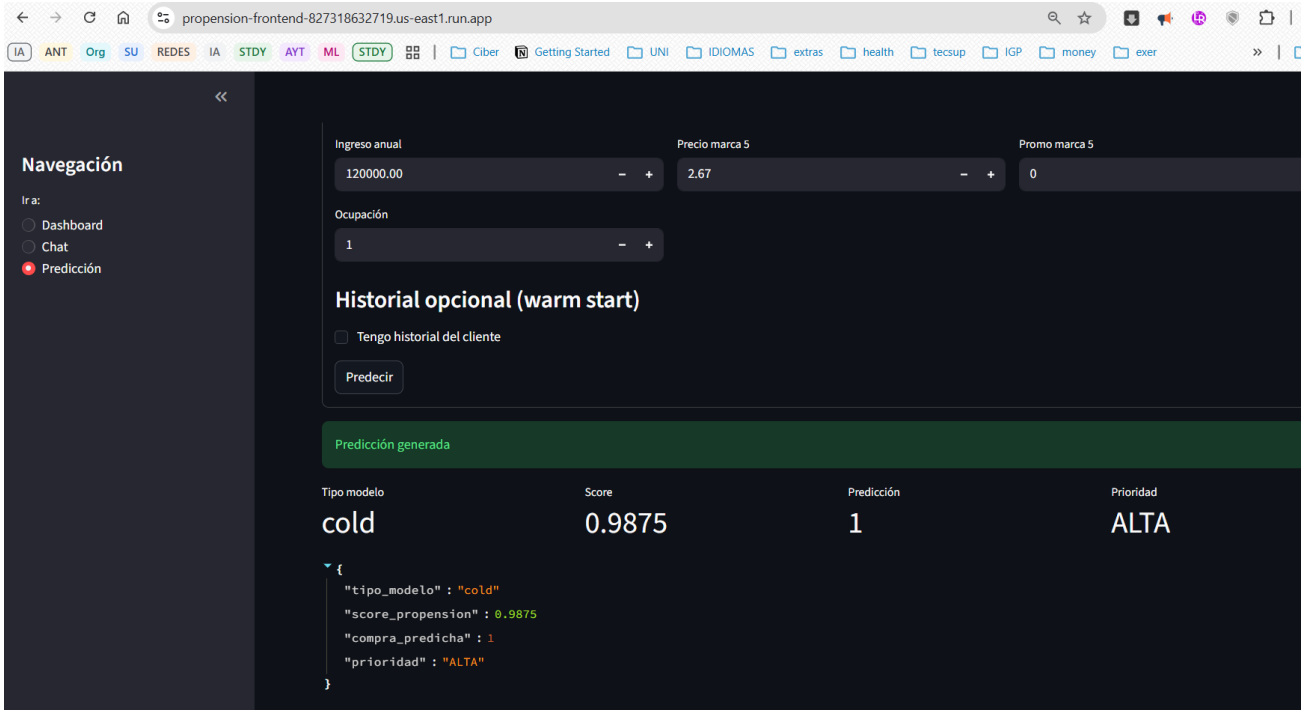
Tengo un cliente nuevo, hombre casado de 60 años, ingreso alto. ¿Lo llamo?	predecir_cliente	Cold-Start
Un cliente con 30 visitas que compra el 70% del tiempo, ¿qué prioridad tiene?	predecir_cliente	Warm-Start
Clientes segmento Medio-Alto con score mayor a 0.6	query_scores	BigQuery



	id_cliente	segmento	score_propension	compra_predicha	tasa_compra_historica	dias_desde_ultima_compra	frecuencia_compras_30d	bucket	prioridad	canal_sugerido
0	200000214	Bajo	0.1125	0	0.1458	268	0	D01	BAJA	EMAIL
1	200000068	Bajo	0.1057	0	0.1613	268	0	D01	BAJA	EMAIL
2	200000274	Bajo	0.1113	0	0.1456	268	0	D01	BAJA	EMAIL
3	200000394	Bajo	0.1119	0	0.136	228	0	D01	BAJA	EMAIL
4	200000013	Bajo	0.0943	0	0.1078	204	0	D01	BAJA	EMAIL



	id_cliente	segmento	score_propension	compra_predicha	tasa_compra_historica	dias_desde_ultima_compra	frecuencia_compras
0	200000247	Alto	0.8681	1	0.9249	1	
1	200000393	Medio-Alto	0.8403	1	0.7247	14	
2	200000091	Medio-Alto	0.8181	1	0.731	14	
3	200000318	Medio-Alto	0.8178	1	0.6598	11	
4	200000475	Medio-Alto	0.8176	1	0.5772	10	
5	200000345	Medio-Alto	0.813	1	0.774	14	
6	200000297	Medio-Alto	0.8127	1	0.7079	14	
7	200000459	Medio-Alto	0.8107	1	0.6033	10	
8	200000443	Medio-Alto	0.8022	1	0.5656	10	
9	200000489	Medio-Alto	0.8019	1	0.6111	7	


propension-frontend-827318632719.us-east1.run.app
Navegación
Ir a:
Dashboard
Chat
Predicción
Ingreso anual
120000.00
Precio marca 5
2.67
Promo marca 5
0
Ocupación
1
Historial opcional (warm start)
Tengo historial del cliente
Predecir
Predicción generada
Tipo modelo
cold
Score
0.9875
Predicción
1
Prioridad
ALTA
{
"tipo_modelo" : "cold"
"score_propension" : 0.9875
"compra_predicha" : 1
"prioridad" : "ALTA"
}

8. Evaluación del Impacto en el Negocio y Captura de Valor

8.1 La pregunta de negocio que responde el modelo

El modelo no responde '¿cuántos clientes comprarán?' sino '¿a cuál de mis clientes debo contactar para maximizar el retorno de cada contacto?'. No se usa como una decisión binaria sino como un sistema de priorización: ordenar clientes de mayor a menor propensión y activar canales según el score.

8.2 Estrategia por canal comercial

Canal	Umbral score	Tasa en grupo	Recall	Uplift vs aleatorio
Email / Notificación push	≥ 0.35	32.3%	74%	+28%
Alerta CRM / vendedor	≥ 0.50	41.7%	50%	+65%
Llamada comercial	≥ 0.60	46.5%	37%	+84%

> *Interpretación: si el equipo comercial llama solo a los 1,649 clientes con score ≥ 0.60, obtiene 84% más conversiones por llamada. Cada llamada es 1.84 veces más productiva que sin el modelo.*

8.3 Tabla de deciles, Lift Analysis

Decil	N	Tasa de compra	Lift	Recall acum.
D10 (mayor score)	814	58.8%	2.34x	23%
D09	814	34.6%	1.38x	37%
D08	813	32.1%	1.27x	50%
D07	814	26.5%	1.05x	60%
Baseline (sin modelo)	—	25.2%	1.00x	—

8.4 KPIs clave de captura de valor

KPI	Valor	Descripción
AUC-ROC (test temporal)	0.685	Poder discriminatorio en condiciones reales de producción
Lift decil 10	2.34x	Contactar el top 10% es 2.3x más efectivo que contacto aleatorio
Precision umbral 0.50	41.7% vs 25.1% basal	Calidad del contacto comercial con CRM/alerta
Uplift llamadas	+84%	ROI incremental de la fuerza de ventas

Estabilidad	±0.0006 AUC	Reproducibilidad garantizada en cada reentrenamiento
Clientes ALTA prioridad	72 de 500 (14.4%)	Candidatos para llamada esta semana
Costo mensual GCP	~$7/mes	Cubierto por créditos GCP de $300

8.5 Áreas de impacto en el negocio

- **Ventas: fuerza de ventas concentra esfuerzo en clientes con señal de compra. 1 llamada del modelo = 2.34 llamadas aleatorias en resultados.**
- **Marketing: segmentación precisa permite personalizar mensajes por canal. ALTA → llamada, MEDIA → CRM, BAJA → email de bajo costo.**
- **Inventario: scores agregados por día anticipan la demanda, optimizando abastecimiento.**
- **Retención: clientes de segmento Medio-Alto o Alto con score decreciente señalan riesgo de fuga antes de que ocurra.**
- **Eficiencia operativa: el agente de IA reduce de horas a segundos el tiempo para obtener la lista de prioridades del día.**
- **Escalabilidad: arquitectura cloud-native permite incorporar nuevos clientes, actualizar scores diariamente y reentrenar mensualmente.**

9. Roadmap y Mejoras Futuras

Corto plazo

- Endurecer validación SQL del planner para mayor seguridad.
- Agregar logging estructurado en el backend.
- Automatizar el reentrenamiento mensual si el AUC cae por debajo de 0.65.

Mediano plazo

- Evolucionar el agente hacia tool calling nativo más sofisticado.
- Agregar memoria conversacional controlada.
- Versionar modelos formalmente con MLOps (Vertex AI Model Registry).
- Enriquecer el frontend con trazabilidad de consultas y visualizaciones adicionales.

Largo plazo

- Conectar resultados a CRM y campañas de contacto para medir uplift real post-implementación.
- Monitorear deriva del modelo con alertas automáticas.
- Incorporar señales externas: canal de contacto preferido, comportamiento en otros canales.

10. Conclusiones

Este proyecto no se limitó a generar un score. Se diseñó y construyó una solución integral que combina machine learning, analítica, servicios cloud y un agente conversacional útil para negocio, todo desplegado y accesible en producción.

- **Metodología honesta: AUC 0.685 con backtesting rolling es el número real de producción. HistGBM obtenía 0.82 con split aleatorio pero caía a 0.67 en test temporal, fue descartado con criterio.**
- **Arquitectura de dos modelos: cold-start/warm-start resuelve el problema del cliente nuevo de forma rigurosa. La tasa basal del 10.6% en primera visita valida que son dos problemas distintos.**
- **Valor de negocio cuantificado: lift 2.34x y uplift +84% son indicadores directamente accionables.**
- **Solución completamente desplegada: backend FastAPI + frontend Streamlit + BigQuery + Gemini en Cloud Run, accesibles en las URLs indicadas al inicio del documento.**
- **Explicabilidad garantizada: Permutation Importance y SHAP confirman que el modelo captura señales reales, no artefactos.**
- **Cierre del ciclo completo: datos → modelo → API → UI → Docker → Cloud Run. Exactamente lo que distingue un proyecto desplegable de un notebook.**